RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Sunday, 30 August 2026, 8:10 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:11 PM
Time taken	41 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for sorting in ascending order
int compareAsc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Comparison function for sorting in descending order
int compareDesc(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main() {
    int n;

    // Read the size of the arrays
    if (scanf("%d", &n) != 1) return 0;

    int *arrayOne = (int*)malloc(n * sizeof(int));
    int *arrayTwo = (int*)malloc(n * sizeof(int));

    // Read elements for the first array
    for (int i = 0; i < n; i++) {
        scanf("%d", &arrayOne[i]);
    }

    // Read elements for the second array
    for (int i = 0; i < n; i++) {
        scanf("%d", &arrayTwo[i]);
    }

    // Sort the first array in ascending order
    qsort(arrayOne, n, sizeof(int), compareAsc);

    // Sort the second array in descending order
    qsort(arrayTwo, n, sizeof(int), compareDesc);

    // Calculate the minimum sum of products
    long long minSum = 0;
    for (int i = 0; i < n; i++) {
        minSum += (long long)arrayOne[i] * arrayTwo[i];
    }

    // Print the final result
    printf("%lld\n", minSum);

    // Free allocated memory
    free(arrayOne);
    free(arrayTwo);

    return 0;
```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course